

Reproduction and Critical Analysis of a DSP Research Study

Back to Basics: Fast Denoising Iterative Algorithm

Student: Tamjid Tahmid

Student ID: 20230105012

Department: Electrical and Electronic Engineering, AUST

Course: EEE 3218 - Digital Signal Processing Lab

Selected paper: Deborah Pereg, "Back to basics: Fast denoising iterative algorithm," *Signal Processing*, vol. 221, 109482, 2024.

Reproduction scope: Natural-image AWGN denoising using the BTB iterative update. The original paper uses pretrained TNRD as the base denoiser; this implementation uses Non-Local Means (NLM) to provide a lightweight, explainable Python reproduction.

Abstract

This project reproduces the central iterative idea of the Back-to-Basics (BTB) image-denoising framework proposed by Pereg (2024). A user-selected image is converted to grayscale, resized and normalized, then corrupted with additive white Gaussian noise (AWGN). Non-Local Means (NLM) is used as the base denoiser inside the BTB update. The reconstruction is evaluated using mean squared error (MSE), peak signal-to-noise ratio (PSNR) and structural similarity (SSIM). Sensitivity experiments study the effect of AWGN level and the number of BTB iterations. The selected showcase image demonstrates an important critical result: visible smoothing does not necessarily improve PSNR. Repeated NLM-based BTB iterations can over-smooth high-frequency detail, showing that BTB performance depends strongly on the base denoiser and parameter choices.

1. Introduction

Image denoising is a two-dimensional digital signal processing problem in which an estimate of an unknown clean image is recovered from a corrupted observation. The selected paper proposes BTB, a simple iterative framework that repeatedly applies a denoiser and moves the current estimate toward the denoised result. The paper studies AWGN, Poisson noise and speckle interference. This OEL focuses on the natural-image AWGN case because it is easy to visualize and allows direct comparison of unprocessed and processed signals.

2. Reproduced Segment and Algorithm

The BTB update implemented here is:

$$\begin{aligned}z_{_}(t+1) &= f(x_{_}t) \\x_{_}(t+1) &= (1 - \mu)x_{_}t + \mu z_{_}(t+1)\end{aligned}$$

Here $x_{_}t$ is the current image estimate, $f(\cdot)$ is the base denoiser, μ is the update weight, and t is the iteration index. In this reproduction, $f(\cdot)$ is NLM, $\mu = 0.7$, and the main demonstration uses $T = 3$ iterations.

Processing sequence

User-selected image -> grayscale -> 256 x 256 resize -> normalization to [0,1] -> AWGN addition -> NLM base denoising -> BTB weighted update -> MSE/PSNR/SSIM evaluation -> sensitivity analysis.

3. Implementation Details

Item	Setting
Language	Python 3
Libraries	NumPy, Matplotlib, Pillow, scikit-image, Tkinter
Main AWGN level	sigma = 15 on 0-255 scale
Sensitivity AWGN levels	10, 15, 25
BTB iterations	T = 3 for main demonstration
BTB weight	mu = 0.7
NLM strength	$h = 0.8 * (\text{sigma} / 255)$
NLM patch size	5
NLM patch distance	6
Image size	256 x 256 grayscale
Random seed	1 (reproducible noise)

The original paper uses pretrained TNRD for its natural-image experiments. NLM is substituted here so that the implementation remains self-contained and easy to explain. This difference means the numerical results are not expected to reproduce the paper exactly.

4. Results and Analysis

Figure 1 compares the selected reference image, the AWGN-corrupted input, and the BTB + NLM output.

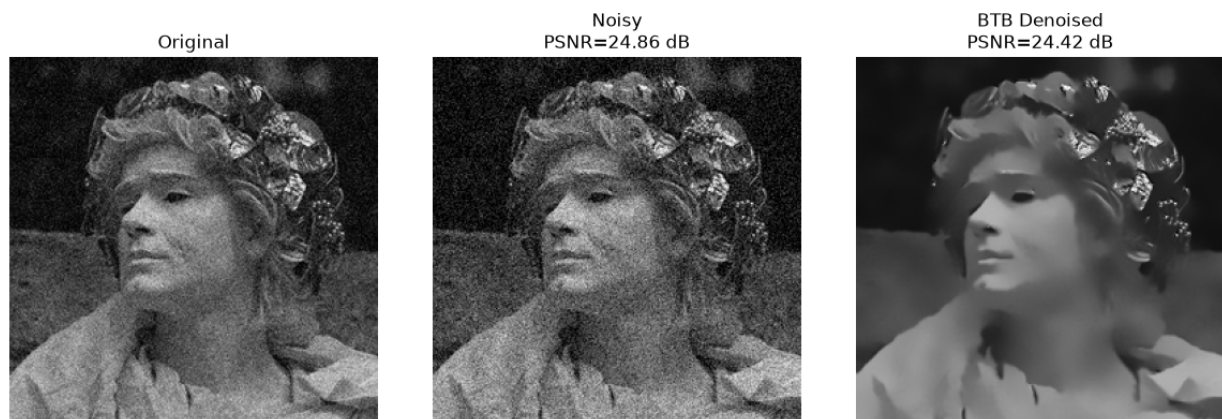


Figure 1. Original, noisy and BTB-denoised image.

Condition	PSNR
Noisy input	24.86 dB
BTB + NLM output	24.42 dB

The processed image looks smoother, but PSNR decreases slightly. This is not a contradiction: PSNR measures pixel fidelity to the selected reference, not subjective smoothness. The reference image itself contains substantial fine texture and grain-like high-frequency content. NLM suppresses part of that content together with added AWGN, so the result can look cleaner while becoming numerically less similar to the reference.

4.1 Noise-level sensitivity

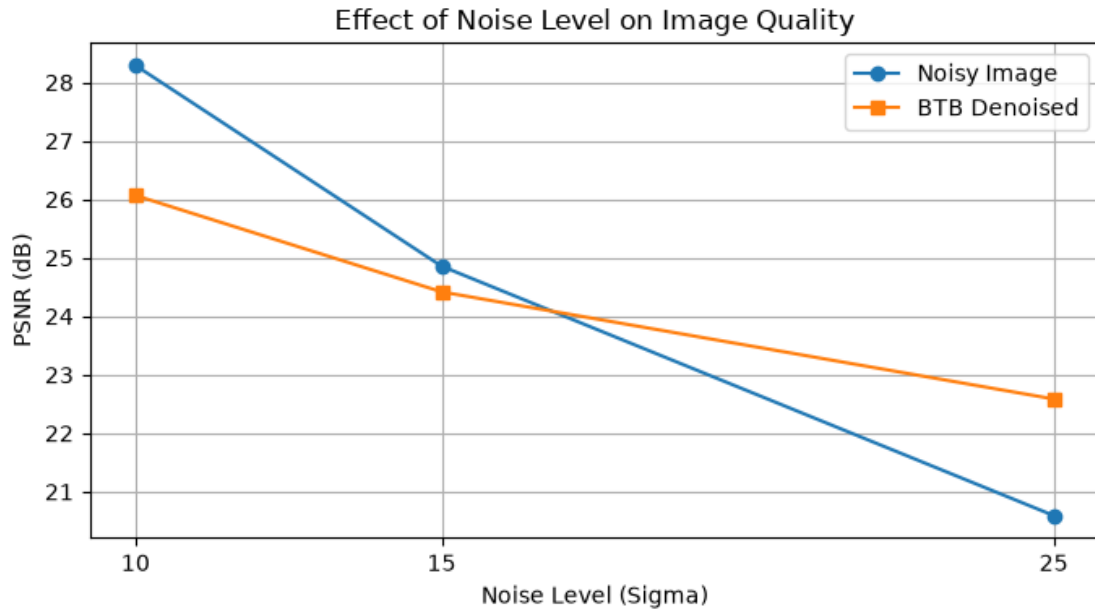


Figure 2. PSNR versus AWGN level for noisy input and BTB + NLM output.

At sigma = 10 and 15, the fixed NLM/BTB settings reduce PSNR relative to the noisy input, indicating over-smoothing. At sigma = 25, the BTB output has a higher PSNR than the noisy input, showing that the same denoising configuration becomes more useful when the corruption is stronger. The experiment therefore demonstrates parameter dependence rather than universal improvement.

4.2 Iteration sensitivity

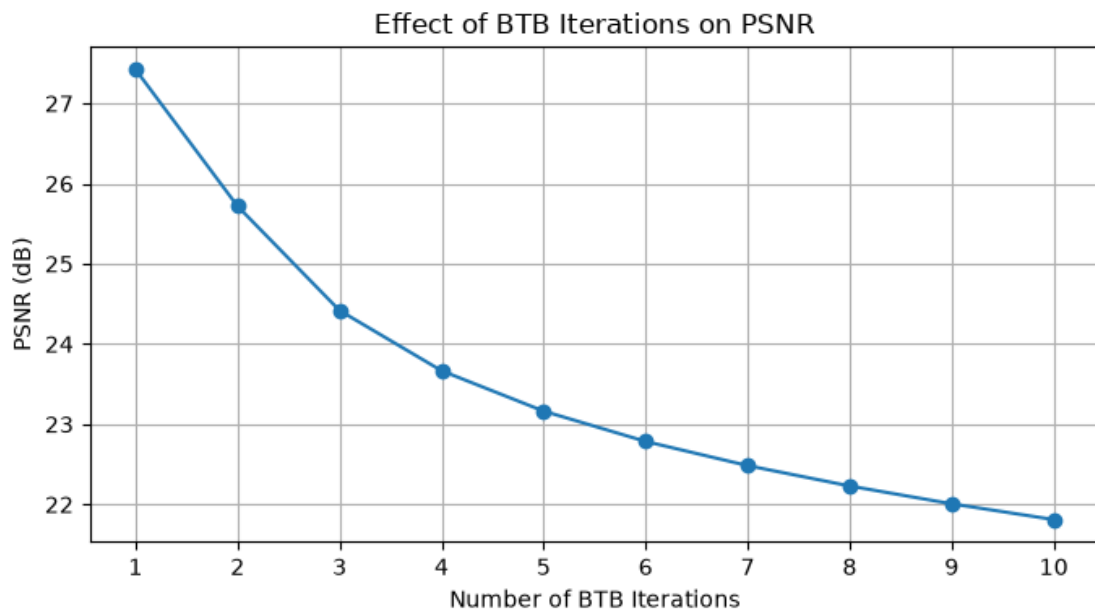


Figure 3. PSNR versus BTB iteration count at sigma = 15.

For this image and NLM configuration, PSNR is highest at the first tested iteration and decreases as more iterations are applied. Repeated NLM denoising progressively removes high-frequency detail. This result is useful for critical evaluation because it shows that increasing the number of BTB iterations is not automatically beneficial.

5. Critical Evaluation

What is reproduced successfully? The implementation preserves the central BTB iterative structure, controlled AWGN corruption, direct visual comparison, reference-based quality metrics, and parameter-sensitivity analysis.

Main deviation from the paper: The original natural-image experiment uses pretrained TNRD; this reproduction uses NLM. Consequently, this work should be described as a method-level reproduction rather than an exact numerical replication.

Key finding: A strong denoiser and repeated iterations can over-smooth the image. The noise-level graph shows that the selected parameters are more useful under stronger corruption, while the iteration graph shows degradation with repeated processing for the selected image.

Possible improvements: tune NLM parameters for each noise level; estimate the noise level from the input; compare NLM with other base denoisers; use a validation rule or stopping criterion; test multiple standard images; and reproduce the original TNRD setup if the pretrained model and exact environment are available.

6. Software and Reproducibility

The program provides a file-selection dialog so any JPG, JPEG, PNG or BMP image can be tested. A fixed NumPy random seed is used when adding AWGN, so the same image and sigma generate the same noise realization. Required Python packages are listed in requirements.txt in the accompanying GitHub repository.

7. Conclusion

The project demonstrates the BTB iterative denoising concept in a compact Python implementation. The experiment confirms that the BTB framework is easy to implement but its success depends on the behavior of the base denoiser and the chosen parameters. The most important experimental lesson is that visual smoothing, PSNR improvement and iteration count are not equivalent objectives. Sensitivity analysis is therefore essential when evaluating iterative denoising systems.

References

- [1] D. Pereg, "Back to basics: Fast denoising iterative algorithm," *Signal Processing*, vol. 221, 109482, 2024. DOI: 10.1016/j.sigpro.2024.109482.
- [2] A. Buades, B. Coll, and J.-M. Morel, "Non-local means denoising," *Image Processing On Line*, vol. 1, pp. 208-212, 2011.